Object Oriented Programming (Classes-II)

Abbas Khalid

October 20, 2025

Table of contents

Last Lecture

```
1  class Box{
2     double length, breadth, height;
3   public:
4     Box (double l, double b, double h){
5       length = l;
6       breadth = b;
7       height = h;
8      }
9     double volume (){return length*breadth*height;}
10  };
11
12 int main(){
13
14   Box mybox (10.0,10.0,10.0);
15   double vol = mybox.volume();
16   cout<<"Volume = "<<vol<<endl;
17
18 return 0;
19  }
```

```
Volume = 1000
```

Last Lecture (Cont.)

```
1  class Box{
2    double length, breadth, height;
3   public:
4    Box(double, double, double);
5    double volume (){return length*breadth*height;}
6  };
7
8  int main(){
9    Box mybox (10.0,10.0,10.0);
10   double vol = mybox.volume();
11   cout<<"Volume = "<<vol<<endl;
12
13 return 0;
14 }
15
16 Box::Box(double l, double b, double h){
17       length = l;   breadth = b;  height = h;
18 }
```

```
Volume = 1000
```

Constructor Overloading

- Defining two or more constructors with same name but different parameter-list within the same class is called constructor overloading.
- The *argument – list* is used to determine which version of the overloaded constructor is called.

Example: Constructor Overloading

```
Volume = -1
Volume = 343
Volume = 1000
```

```
1  class Box{
2      double length, breadth, height;
3    public:
4      Box(){
5          length = breadth = height = -1;
6      }
7
8      Box(double len){
9          length = breadth = height = len;
10     }
11
12     Box(double l, double b, double h){
13          length = l;
14          breadth = b;
15          height = h;
16          }
17
18     double volume(){return length*breadth*height;}
19   };
```

Example: Constructor Overloading (Cont.)

```
1  int main(){
2
3    Box b1;
4    Box b2 (7.0);
5    Box b3 (10.0,10.0,10.0);
6    double vol;
7    vol = b1.volume();
8    cout<<"Volume = "<<vol<<endl;
9    vol = b2.volume();
10   cout<<"Volume = "<<vol<<endl;
11   vol = b3.volume();
12   cout<<"Volume = "<<vol<<endl;
13
14 return 0;
15  }
```

Default Argument Values

- Just like ordinary functions, we can specify the default parameters for class member functions as well as constructors.
- If we put the definition of the member function inside the class definition, then we can put the default values for the parameters in the function header.
- If we include the declaration of a function in the class definition, then the default parameter values should go in the declaration and not in function definition.
- Same is true for constructors.
- Note that, the constructor with all default parameter values also serves as a default constructor. If we include both of them in our program at the same time, it will be an error.

Example 1: Use of Default Argument Values

```
1  class Box{
2     int length, breadth, height;
3   public:
4     Box(int l = 1, int b = 1, int h = 1);
5     int volume ();
6  };
7
8  int main(){
9    Box b1, b2(10), b3(10,10), b4(10,10,10);
10   cout<<"Volume = "<<b1.volume()<<endl;
11   cout<<"Volume = "<<b2.volume()<<endl;
12   cout<<"Volume = "<<b3.volume()<<endl;
13   cout<<"Volume = "<<b4.volume()<<endl;
14
15 return 0;
16 }
17 Box::Box (int l, int b, int h){
18   length = l; breadth = b;  height = h;
19   }
20 int Box::volume (){return length*breadth*height;}
```

```
Volume = 1
Volume = 10
Volume = 100
Volume = 1000
```

Example 2: Illegal Use of Default Argument Values

```
1  class Box{
2     int length, breadth, height;
3   public:
4     Box(){int length = -1, breadth = -1, height = -1;}
5     Box(int l = 1, int b = 1, int h = 1);
6     int volume ();
7  };
8
9  int main(){
10    Box b1;    // Ambiguous Call
11    cout<<"Volume = "<<b1.volume()<<endl;
12
13 return 0;
14 }
15 Box::Box (int l, int b, int h){
16    length = l; breadth = b;  height = h;
17    }
18 int Box::volume (){return length*breadth*height;}
```

Example 3: Illegal Use of Default Argument Values

```
1  class Box{
2      int length, breadth, height;
3   public:
4      Box(int l){int length = l, breadth = l, height = l;}
5      Box(int l = 1, int b = 1, int h = 1);
6      Box(int l , int b = 1, int h = 1);
7  };
8
9  int main(){
10     Box b1(10);    // Ambiguous Call
11
12 return 0;
13 }
14 Box::Box (int l, int b, int h)
15  {length = l; breadth = b;  height = h;}
16 Box::Box (int l, int b, int h)
17  {length = l; breadth = b;  height = h;}
```

The *explicit* Keyword

- When we have a constructor that requires only one argument, we can either use ob(i) or ob=i to initialize an object *ob* with *i*. The reason is that whenever we create a constructor that takes one argument, we are also implicitly creating a conversion from the type of that argument to the type of the class.
- Sometimes we do not want this automatic conversion to take place. For this purpose, C++ defines the keyword *explicit*. The *explicit* specifier applies only to constructors. A constructor specified as *explicit* will only be used when an initialization uses the normal constructor syntax. It will not perform any automatic conversion.

Example 1 : Implicit Conversion

```
1  class myClass{
2    int a;
3   public:
4    myClass(int x){a = x;}
5    int getA(){return a;}
6   };
7
8  int main(){
9
10   myClass ob = 10;   // Changes to myClass(10)
11   cout<<"Value = " <<ob.getA();
12
13 return 0;
14 }
```

```
Value = 10
```

Example 2 : Illegal Conversion

```
1  class myClass{
2    int a;
3   public:
4    myClass(int x){a = x;}
5    int getA(){return a;}
6   };
7
8  int main(){
9
10   myClass ob;    // Incorrect
11   ob = 10;    // Incorrect
12   cout<<"Value = " <<ob.getA();
13
14 return 0;
15 }
```

Example 3 : Use of *explicit* Keyword

```
1  class myClass{
2    int a;
3   public:
4    explicit myClass(int x){a = x;}
5    int getA(){return a;}
6   };
7
8  int main(){
9
10   myClass ob = 10;   // Conversion not allowed
11   cout<<"Value = " <<ob.getA();
12
13 return 0;
14 }
```

Example 4 : Manual Conversion

```
Value = 10
```

```
1  class myClass{
2    int a;
3   public:
4    explicit myClass(int x){a = x;}
5    int getA(){return a;}
6   };
7
8  int main(){
9
10   myClass ob = (myClass)10;   // Changes to myClass(10)
11   cout<<"Value = " <<ob.getA();
12
13 return 0;
14 }
```

Member-wise Copy using Assignment Operator

- The assignment operator (=) can be used to assign an object to another object of the same type.
- Such assignment is by default performed by member wise copy i.e., each member of an object is copied (assigned) individually to the same member in another object.

Example: Member-wise Copy using Assignment Operator

```
1  class Box{
2    int length, breadth, height;
3   public:
4    Box(int l = 1, int b = 1, int h = 1);
5    int volume ();
6  };
7
8  int main(){
9    Box b1(5,5); Box b2;
10   cout<<"Volume = "<<b1.volume()<<endl;
11   cout<<"Volume = "<<b2.volume()<<endl;
12   b2 = b1;     // member-wise copy
13   cout<<"Volume = "<<b2.volume()<<endl;
14
15 return 0;
16 }
17 Box::Box (int l, int b, int h){
18   length = l; breadth = b;  height = h;
19 }
20 int Box::volume (){return length*breadth*height;}
```

```
Volume = 25
Volume = 1
Volume = 25
```

The Copy Constructor

- We know that when we have no constructor defined, the compiler provides a default constructor to allow an object to be created.
- Similarly, the compiler facilitates us with a default version of a copy constructor. This default copy constructor creates a new object by copying the existing object member by member. The general form of a copy constructor is:

```
1 className(const className& Obj){
2       //   body of the constructor
3 }
```

Here, *obj* is a reference to the object. The *const* qualifier prevents modifications to the original object while passing by reference (&) avoids unnecessary copying of the object's data.

- Omitting *const* is not a syntax error but not recommended. Omitting & is a syntax error.

Whenever, we pass an object as an argument to a function, the compiler will make a copy of the object by calling the default copy constructor. We do not need to call copy constructor explicitly.

Example 1: Default Copy Constructor

```
1  class Box{
2    int length, breadth, height;
3   public:
4    Box(int l = 1, int b = 1, int h = 1);
5    int volume (){return length*breadth*height;}
6  };
7
8  int main(){
9    Box b1(5,5);
10   Box b2(b1); // or Box b2 = b1;
11   cout<<"Volume = "<<b1.volume()<<endl;
12   cout<<"Volume = "<<b2.volume()<<endl;
13
14 return 0;
15 }
16 Box::Box (int l, int b, int h){
17   length = l; breadth = b;  height = h;
18 }
```

```
Volume = 25
Volume = 25
```

Example 2: User-defined Copy Constructor

```
class Box{
  int length, breadth, height;
 public:
  Box(int l = 1, int b = 1, int h = 1);
   Box(const Box& b){cout<<"Copy Constructor.."<<endl;}
  int volume (){return length*breadth*height;}
};

int main(){

  Box b1(5,5);
  Box b2(b1);   // Default no longer available
  cout<<"Volume = "<<b1.volume()<<endl;
  cout<<"Volume = "<<b2.volume()<<endl;
return 0;
}
Box::Box (int l, int b, int h)
  {length = l; breadth = b;   height = h;}
```

```
Copy Constructor ..
Volume = 25
Volume = 0
```

Example 3: User-defined Copy Constructor

```
1  class Box{
2    int length, width, height;
3   public:
4    Box(int l = 1, int b = 1, int h = 1);
5    Box(const Box& b){length = b.length; width = b.width; height = b.height;}
6    int volume (){return length*width*height;}
7  };
8
9  int main(){
10   Box b1(5,5);
11   Box b2(b1);   // or   Box b2 = b1;
12   cout<<"Volume = "<<b1.volume()<<endl;
13   cout<<"Volume = "<<b2.volume()<<endl;
14
15 return 0;
16 }
17 Box::Box (int l, int b, int h){
18   length = l; width = b;  height = h;
19 }
```

```
Volume = 25
Volume = 25
```

Exercise: What will be the Output?

```
1  class Test {
2    int data;
3   public:
4     // Copy constructor without const and &
5    Test(Test other) {    // Recursion
6           data = other.data;
7      }
8    void setData(int d){data = d;}
9    int getData(){return data;}
10 };
11
12 int main() {
13
14   Test t1;
15   t1.setData(10);
16   Test t2 = t1;    cout<<t2.getData()<<endl;
17   Test t3(t1);     cout<<t3.getData()<<endl;
18 return 0;
19 }
```

Remarks

- The copy constructor is used to create a new object based on an existing object when the new object is declared and initialized.

```
1 MyClass obj1;  // Create an object
2 MyClass obj2 = obj1;  // Copy constructor
```

- The assignment operator is used to copy the values of one object into another object that already exists.

```
1 MyClass obj1;  // Create an object
2 MyClass obj2;  // Create another object
3 obj2 = obj1;  //  Assignment operator
```

- If the copy constructor is defined without the & (i.e., taking its argument by value), it would lead to infinite recursion because calling the copy constructor itself creates a copy of the object, which requires calling the copy constructor again, leading to an infinite loop.

Destructors

- A class destructor is a class member that destroys an object when it goes out of scope or when a dynamically created object is deleted from the free store.
- The destructor for an object is called automatically whenever the object should be destroyed. Usually, there is no need to call destructor explicitly but it is often necessary to define it.
- A class destructor is a public member function with the same name as the class, preceded by a tilde (~) sign. The destructor does not have any parameter and it does not return a value.
- The general form for a destructor is:

```
1 ~className (){
2    //code to destroy objects
3 }
```

Destructors

- A class can only have one destructor regardless the number of constructors it has. If we do not define a destructor for a class, the compiler will always supply a *public* and *inline* destructor.
- A local object's constructor is executed when the object's declaration statement is encountered. The destructor functions for local objects are executed in the reverse order of the constructor functions.
- A global object's constructor executes before *main* begins the execution. Global constructors are executed in order of their declaration, within the same file. We cannot know the order of execution of global constructors spread among several files. Global destructors execute in reverse order after *main* has terminated.

Example 1

```
1  class Cat{
2    int age;
3   public:
4    Cat(int initialAge){age = initialAge;}
5    ~Cat(){ }
6    int getAge(){return age;}
7    void setAge(int a){age = a;}
8  };
9
10 int main(){
11   Cat mano(5);
12   cout<<"Mano is "<<mano.getAge()<<" years old."<<endl;
13   mano.setAge(7);
14   cout<<"Now Mano is "<<mano.getAge()<<" years old."<<endl;
15
16 return 0;
17 }
```

```
Mano is 5 years old.
Now Mano is 7 years old.
```

Example 2

```
class Destructor{
   int who;
  public:
   Destructor(int id){
     who = id; cout<<" Initializing: "<<who<<endl;}

   ~Destructor()
     {cout<<"Destructing:"<<who<<endl;}

} global1(1), global2(2);

int main(){

  Destructor local1(3);
  cout<<" Hello....."<<endl;
  Destructor local2(4);

return 0;
}
```

```
Initializing: 1
Initializing: 2
Initializing: 3
Hello.....
Initializing: 4
Destructing: 4
Destructing: 3
Destructing: 2
Destructing: 1
```

Passing Objects through Functions

- Objects are passed to functions in the same way as any other type of variable is passed i.e., object may be passed both ways: through call-by-value or through call-by-reference mechanism.
- In the call by value mechanism, when a copy of an object is created to be used as an argument to a function, the normal constructor is not called. Instead, the default copy constructor makes a bit-by-bit identical copy. However, when the copy is destroyed (usually by going out of scope when the function returns), the destructor is called.
- When the object is passed to the function by reference, the called function refers to the original object with an alias name; it does not make a copy of the passed object. Thus, no constructor is invoked since no new object has been created. Also, when the called function is terminated, no destructor is invoked since there does not exist any copy of the object which is to be destroyed.
- A function may return an object to the caller just like any other variable is returned by the function.

Example: Call by Value

```
class ByValue{
    int x;
  public:
   ByValue(int i) {
        x = i;
        cout<<"Construct Object: "<< x <<endl;
   }
    ~ByValue() {
       cout<<"Destroy Object: "<< x <<endl;
    }
    ByValue(const ByValue& obj){
         cout<<"Copy Object: "<< obj.x <<endl;
      }
    void set(int i){x = i;}
    int get(){return x;}
};
```

Example: Call by Value (Cont.)

```
1  void func(ByValue v){
2      v.set(2) ;
3      cout<<"Inside  func(). "<<endl;
4      cout<<"x = "<<v.get()<<endl;
5  }
6
7  int main(){
8
9      ByValue V(1);
10     cout<<"Inside main(). "<<endl;
11     cout<<"x = "<<V.get()<<endl;
12     func(V);        // Call by value
13     cout<<"Back in main(). "<<endl;
14     cout<<"x = "<<V.get()<<endl;
15
16 return 0;
17 }
```

```
Construct Object: 1
Inside main().
x = 1
Copy Object:  1
Inside  func().
x = 2
Destroy Object:  2
Back in main().
x = 1
Destroy Object:  1
```

Example: Call by Reference

```
1  class ByRef{
2      int x;
3    public:
4      ByRef(int i) {
5          x = i;
6          cout<<"Construct Object: "<< x <<endl;
7          }
8
9      ~ByRef() {cout<<"Destroy Object: "<< x <<endl;}
10
11     ByRef(const ByRef& obj){
12       cout<<"Copy Object: "<< obj.x <<endl;
13      }
14
15     void set(int i){x = i;}
16     int get(){return x;}
17 };
```

Example: Call by Reference (Cont.)

```
1  void func(ByRef& v){
2      v.set(2) ;
3      cout<<"Inside  func(). "<<endl;
4      cout<<"x = "<<v.get()<<endl;
5  }
6
7  int main(){
8
9      ByRef V(1);
10     cout<<"Inside main(). "<<endl;
11     cout<<"x = "<<V.get()<<endl;
12     func(V);        // Call by Reference
13     cout<<"Back in main(). "<<endl;
14     cout<<"x = "<<V.get()<<endl;
15
16 return 0;
17 }
```

```
Construct Object: 1
Inside main().
x = 1
Inside  func().
x = 2
Back in main().
x = 2
Destroy Object:  2
```

Exercise 1: What will be the Output?

```
1  class Test{
2     int x;
3   public:
4     Test(int a = 0)
5       {x = a; cout<<"Construct Object."<<endl;}
6     ~Test() {cout<<"Destroy Object."<<endl;}
7
8     Test (const Test& obj){cout<<"Copy Object."<<endl;}
9
10    void func(Test t1, Test& t2)
11      {t1.x += 5; t2.x += 5;}
12
13    int getX(){return x;}
14 };
15 int main(){
16    Test t1 (5),t2(10),t;
17    t.func(t1,t2);
18    cout<<"Result = ";
19    cout<<t1.getX()+t2.getX()<<endl;
20 return 0;
21 }
```

```
Construct Object.
Construct Object.
Construct Object.
Copy Object.
Destroy Object.
Result = 20
Destroy Object.
Destroy Object.
Destroy Object.
```

Exercise 2: What will be the Output?

```
1  class Test{
2     int x;
3   public:
4     Test(int a = 0)
5       {x = a; cout<<"Construct Object."<<endl;}
6     ~Test() {cout<<"Destroy Object."<<endl;}
7
8     Test (const Test& obj){cout<<"Copy Object."<<endl;}
9
10    void func(Test& t1, Test& t2)
11      {t1.x += 5; t2.x += 5;}
12
13    int getX(){return x;}
14 };
15 int main(){
16    Test t1 (5),t2(10),t;
17    t.func(t1,t2);
18    cout<<"Result = ";
19    cout<<t1.getX()+t2.getX()<<endl;
20 return 0;
21 }
```

```
Construct Object.
Construct Object.
Construct Object.
Result = 25
Destroy Object.
Destroy Object.
Destroy Object.
```

Returning Objects

- Returning by value: When we return an object by value, C++ makes a copy (or move) of the local object before the function ends. That new copy is what the caller receives (not the original local object). So, even though the local object inside the function is destroyed when the function ends, the returned copy continues to exist safely in the caller's scope. Return Value Optimization (RVO) and Named Return Value Optimization (NRVO) often eliminate these copies automatically, so it's usually efficient.
- Returning by reference: When we return by reference, we are returning an alias (another name) for an existing object. The function does not create a copy. Note that returning a reference to a local object causes undefined behavior, because the object is destroyed when the function ends.

Example: Returning Objects

```
1  class Test{
2     int x;
3   public:
4     Test(int a = 0)
5       {x = a; cout<<"Construct Object."<<endl;}
6
7     ~Test() {cout<<"Destroy Object."<<endl;}
8
9     Test (const Test& obj){cout<<"Copy Object."<<endl;}
10
11    int getX(){return x;}
12 };
13
14 Test fun(){Test t(1); return t;}
15
16 int main(){
17      Test t;
18      t = fun();      // function call
19      cout<<t.getX()<<endl;
20 return 0;
21 }
```

```
Construct Object.
Construct Object.
Destroy Object.
1
Destroy Object.
```

Exercise 1: What will be the output?

```
1  class Test{
2     int x;
3   public:
4     Test(int a = 0) {x = a; cout<<"Construct:"<<x<<endl;}
5     ~Test() {cout<<"Destroy:"<<x<<endl;}
6      Test (const Test& obj){cout<<"Copying..\n";x = obj.x;}
7
8    void setX(int a){x = a;}
9    int getX(){return x;}
10 };
11
12 Test fun(Test t){
13        cout<<"Inside\n"; t.setX(10);
14 return t;
15 }
16
17 int main(){
18      Test t, t1(5);    t = fun(t1);
19      cout<<t.getX()<<endl;
20 return 0;
21 }
```

```
Construct:0
Construct:5
Copying..
Inside
Copying..
Destroy:10
Destroy:10
10
Destroy:5
Destroy:10
```

Exercise 2: What will be the output?

```
1  class Test{
2     int x;
3   public:
4     Test(int a = 0) {x = a; cout<<"Construct:"<<x<<endl;}
5     ~Test() {cout<<"Destroy:"<<x<<endl;}
6      Test (const Test& obj){cout<<"Copying..\n";x = obj.x;}
7
8    void setX(int a){x = a;}
9    int getX(){return x;}
10 };
11
12 Test fun(Test& t){
13        cout<<"Inside\n"; t.setX(10);
14 return t;
15 }
16
17 int main(){
18      Test t, t1(5);    t = fun(t1);
19      cout<<t.getX()<<endl;
20 return 0;
21 }
```

```
Construct:0
Construct:5
Inside
Copying..
Destroy:10
10
Destroy:10
Destroy:10
```

Exercise 3: What will be the output?

```
1  class Test{
2     int x;
3   public:
4     Test(int a = 0) {x = a; cout<<"Construct:"<<x<<endl;}
5     ~Test() {cout<<"Destroy:"<<x<<endl;}
6      Test (const Test& obj){cout<<"Copying..\n";x = obj.x;}
7
8    void setX(int a){x = a;}
9    int getX(){return x;}
10 };
11
12 Test& fun(Test& t){
13        cout<<"Inside\n"; t.setX(10);
14 return t;
15 }
16
17 int main(){
18      Test t, t1(5);    t = fun(t1);
19      cout<<t.getX()<<endl;
20 return 0;
21 }
```

```
Construct:0
Construct:5
Inside
10
Destroy:10
Destroy:10
```

Exercise 4: What will be the output?

```
1  class Test{
2     int x;
3   public:
4     Test(int a = 0) {x = a; cout<<"Construct:"<<x<<endl;}
5     ~Test() {cout<<"Destroy:"<<x<<endl;}
6      Test (const Test& obj){cout<<"Copying..\n";x = obj.x;}
7
8    void setX(int a){x = a;}
9    int getX(){return x;}
10 };
11
12 Test& fun(Test t){
13        cout<<"Inside\n"; t.setX(10);
14 return t;    // Problem! Ref to local object returned
15 }
16
17 int main(){
18     Test t, t1(5);    t = fun(t1);
19     cout<<t.getX()<<endl;
20 return 0;
21 }
```

Initializer List in a Constructor

- An *initializer – list* can also be used to initialize the members of an object.
- The *initializer – list* is specified following the constructor's parameter list.
- It is separated from the parameter list by a colon.
- Each initializer is separated from the next by a comma.
- The general form to specify a constructor *initializer – list* is:

```
className (p1,···,pN):dM1(p1),···,dMN(pN){}
```

- Note that there is no semicolon at the end of the last initializer.
- Here, the values of the data members ($dM1, \cdots, dMN$) are not set using assignment statements in the body of the constructor. The variables $p1, \cdots, pN$ are constructor parameters (local variables).

Initializer List in a Constructor

- When we initialize a data member using assignment statement in the body of the constructor, the data member is first created (using a call to constructor) and then assignment is carried out as a separated operation.
- On the other hand, when we use *initializer – list*, the initial value is used to initialize the data member as it is created. So this is more efficient mechanism than using assignment in the body of the constructor.
- Body of constructor still can be used for initialization purposes.

Example 1: Initializer List

```
1  class Box{
2     double length, breadth, height;
3   public:
4     Box(double l, double b, double h);
5     double volume ();
6  };
7
8  int main(){
9     Box    b1(5.0, 5.0, 5.0);
10    double vol = b1.volume ();
11    cout<<" Volume = "<<vol<<endl;
12 return 0;
13  }
14
15 double Box::volume (){return length*breadth*height;}
```

```
Volume = 125
```

```
Box::Box(double l, double b, double h):
        length (l), breadth (b), height (h) {}
```

Example 2: Initializer List

```
1 class Box{
2    double length, breadth, height;
3  public:
     Box(double l, double b, double h):
           length (l), breadth (b) {height = h;}
   double volume (){return length*breadth*height;}
};

int main(){
    Box    b1(5.0, 5.0, 5.0);
    double vol = b1.volume ();
    cout<<" Volume = "<<vol<<endl;
return 0;
 }
```

```
Volume = 125
```